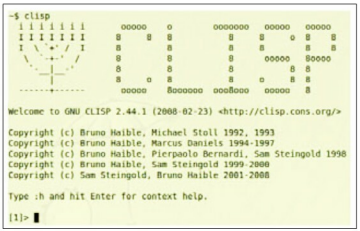

Figure 1: Starting CLISP

In the expression (* 7 (+ 4 3)), the * and the + are called the operator, and the numbers 7, 4, and 3 are called the arguments. In everyday life, we would write this expression as *((4 + 3) * 7)*, but in Lisp, we put the operators first, followed by the arguments, with the whole expression enclosed in parentheses. This is called *prefix* notation, because the operator comes first.

By the way, if you make a mistake and CLISP starts acting crazy, just type *:q* and it'll fix everything. When you want to shut down CLISP, just type *(quit)*.

What's under the hood?

Let's not go down the traditional route of starting with the A B Cs (learning the syntax of the language, its core features, etc). Sometimes, the promise of what lies beneath is more tantalising than baring it all.

Conrad Barski gets you excited about Lisp by showing you how to write a game in it. Let's adopt his method, and write a simple command-line interface game using a Binary Search algorithm. We know that the Binary Search technique continually divides the data in half, progressively narrowing down the search space, until it finds a match, or there are no more items to process. It's the classic guess-my-number game. Ask your friend (or better, your non-technical boss, who yelled at you the last time you fell asleep at a meeting) to pick a number between 1 and 100 (in his head) and your Lisp program would guess it in no more than 7 iterations.

This is how Barski explains the game:

"To create this game, we need to write three functions: *guess-my-number*, *smaller*, and *bigger*. The player simply calls these functions from the REPL. To call a function in Lisp, you put parentheses around it, along with any parameters you wish to give the function. Since these particular functions don't require any parameters, we simply surround their names in parentheses when we enter them."

Here's the strategy behind the game:

1. Determine the upper and lower (big and small) limit of the player's number. In our case, the smallest possible number would be 1, and the biggest would be 100.
2. Guess a number in between these two numbers.
3. If the player says the number is smaller, lower the big limit.
4. If the player says the number is bigger, raise the small limit.
5. We'll also need a mechanism to start over with a different number.

Figure 2: The game in action

In Common Lisp, functions are defined with *defun*, as follows:

```
defun function_name(arguments)
...)
```

As the player calls the functions that make up our game, the program will need to track the small and big limits. In order to do this, we'll need to create two global variables called **small** and **big**. A variable that is defined globally in Lisp is called a *top-level definition*. We can create new top-level definitions with the *defparameter* function.

```
> (defparameter *small* 1)
*SMALL*
> (defparameter *big* 100)
*BIG*
```

The asterisks surrounding the names **big** and **small**—affectionately called *earmuffs*—are completely arbitrary and optional. Lisp sees the asterisks as part of the variable names, and ignores them. Lispers like to mark all their global variables in this way as a convention, to make them easy to distinguish from local variables, which we'll discuss in subsequent articles.

Also, spaces and line breaks are completely ignored when Lisp reads in your code.

Now, the first function we'll define is *guess-my-number*. This uses the values of the **big** and **small** variables to generate a guess of the player's number. The definition looks like what follows:

```
> (defun guess-my-number()
```